

CS 2308: Foundations of Computer Science II

Spring 2026

Section 251

Instructor: Dr. Jill Seaman
Comal 210D
jillseaman@txstate.edu

Office Hours:

Tuesday: 9:00am–10:30am
Thursday: 1:30pm–3:00pm
Friday: 10:30am–12:00pm
in person in my office, zoom by request only

Learning Management System: Class materials will be available on **Canvas**.

Course Format (Mode of Delivery):

This is a **Face to Face** course.

TR 11:00 am - 12:20 pm **Trauth-Huffman Hall 209**

Required Textbook:

We will be using an online textbook from **zyBooks**.

- Please see the Canvas Announcement on how to sign up for and subscribe to your zybooks book. When it asks for a student ID, **use your A0#**!
- If you opt out of BookSmart, the cost is \$99.
- For issues with zybooks: contact their support team: support@zybooks.com.

Required In-Class Response system:

We will use the **Top Hat** classroom response system. You will be able to submit answers to in-class questions using a mobile device or laptop. To register, go to the class Canvas site and click on Top Hat. **Use your A0#** for your student ID. Top Hat is integrated into Canvas and free for Texas State students. There is a mobile app available, in addition to the Top Hat website.

Course Description:

Fundamentals of object-oriented programming. Introduction to abstract data types (ADTs) including lists, stacks, and queues. Searching and sorting. Pointers and dynamic memory allocation. A continuation of CS 1428.

Prerequisites: C or higher in CS 1428: Foundations of Computer Science I

Course Objectives: At the end of the course, the students should be able to:

1. Describe and demonstrate at least two different algorithms for searching and at least two different algorithms for sorting.
2. Implement a divide-and-conquer algorithm to solve an appropriate problem (binary search).
3. State the time/space efficiency of various algorithms (using one of 6 categories of mathematical functions).
4. List the 6 categories of mathematical functions used in analyzing algorithms in order

from slowest to fastest growing.

5. Read and write C++ code that uses pointer variables and memory operations (new, &, *, delete), including pointers to arrays, structures, and objects and the -> operator.
6. Write C++ code that resizes an array using dynamic memory allocation.
7. Write C++ code that deletes dynamically allocated memory to avoid memory leaks.
8. Describe the basic concepts of object-oriented programming.
9. Design, implement, test, and debug simple programs (using objects) in an object-oriented programming language (C++).
10. Describe how the class mechanism supports encapsulation and information hiding.
11. Develop (implement) programs using multiple classes and arrays of objects
12. Develop and use appropriate algorithms, especially for processing lists (insert, remove, search, sort, etc.)
13. Describe structured programming in terms of modules and functions.
14. Develop (implement) programs with source code separated into multiple files, including header (.h) files
15. Create, compile, and run a C++ program in a unix style command-line environment
16. Develop (Implement) C++ programs that create and use simple linked-lists, including code to insert into, delete from, and traverse a linked list structure.
17. Compare and contrast the costs and benefits of dynamic and static data structure implementations.
18. Describe the principle of the Abstract Data Type (ADT) and, in particular, explain the benefits of separation of interface and implementation.
19. Implement user-defined data structures in a high-level language.
20. Implement the list, stack, and queue ADT using arrays and linked lists.
21. Write programs that use each of these data structures: linked lists, stacks, and queues.

Grading:	Participation:	5%	Top Hat
	zyBooks PAs & CAs	5%	lowest of 6 is dropped
	Coding Quizzes	10%	lowest of 7 is dropped
	Programming Assignments:	10%	lowest of 6 is dropped
	Exams:	40%	2 total (20% each)
	Final Exam (comprehensive):	30%	May 7 (Thurs) 11:00am-1:30pm

Exam dates: Mar 5 (Thurs), Apr 28 (Tues).

Letter Grade	Range
A	89.50 to 100%
B	79.50% to 89.49%
C	69.50% to 79.49%
D	59.50% to 69.49%
F	0% to 59.49%

Attendance policy:

Class attendance is very important but it is not directly included as part of your grade.

Class Participation:

Bring a web-enabled device to class to access Top Hat. You will be asked questions in each class (2 points each: 1 for correctness, 1 for any answer). 80% of the total possible points earns full credit for participation. You can miss up to 20% of the points and still get 100%. You may access Top Hat remotely if you are unable to attend class in person.

zyBooks:

There will be assigned readings and activities from the book for each unit. These will be due every other week, starting with Unit 1 at 11:59pm Sunday, Feb 1. Each assignment will include **both** Participation Activities (PAs) and Challenge Activities (CAs).

Coding Quizzes:

These quizzes will occur in class, starting on Feb 3 (see canvas for specific dates). These will be in-class on paper and will occur during the last 15-20 minutes of class. The quiz will require **writing C++ code** or **finding errors in C++ code** or **giving the output produced by C++ code**.

Programming Assignments:

There will be one (large) coding assignment for each unit. The assignments will be given in the zyBooks book. You will develop them using an IDE embedded in a Linux environment from within zyBooks. You will submit and test the programs from within zybooks. In addition to correctness your assignments will be graded for constraints in the instructions and programming style.

Makeup Policy:

Missed Top Hat questions, quizzes, and programming assignments **cannot be made up**. The lowest zybooks assignment, Quiz, and Programming Assignment scores are dropped.

Examinations:

- All Exams will be in class on paper, using pen or pencil, during class time.
- There will be no makeup exams. If you have an instructor-approved excuse, I will replace one missing exam score with your final exam score.

Proposed Course Schedule:

	Topic	Dates
Unit 1	Functions, Arrays & Structs	Jan 22, 27, 29
	Linux	Feb 3
Unit 2	Intro to Classes	Feb 5, 10, 12
Unit 3	Pointers & Dynamic Memory Allocation	Feb 17 19, 24, 26
Unit 4	Searching, Sorting & Analysis	Mar 3, 10, 12, 24
Unit 5	Linked Lists	Mar 26, 31, Apr 2, 7
Unit 6	Stacks & Queues	Apr 9, 14, 16
	List ADT, Review	Apr 21, 23, 30

Exam coverage:

Exam 1 covers Units 1, 2, 3, and Linux.
Exam 2 covers Units 4, 5, and 6.
Final Exam covers Units 1-6, Linux, and List ADT.

Each Exam will have multiple choice questions (50%) and coding questions (50%).

Withdrawals/drops:

You must follow the withdrawal and drop policy set up by the University.
<https://onestop.txst.edu/registrar/dropping-or-withdrawing.html>

Last day to drop with automatic "W": March 31, 2026.

HELP:

Other than the instructor's office hours, there are other places to obtain assistance:

- **CS Lab tutors** (good for help on Programming Assignments): MCS 593 (inside the MCS 590 suite) <https://cs.txst.edu/resources/tutoring.html>
- **CLC tutoring** (good for help on understanding the material): Free walk-in tutoring provided by students in the H-LSAMP Scholars Program.
<https://hlsamp.cose.txst.edu/collaborative-learning-center.html>
- **TA's office hours** see Canvas for times and how to access.

Notifications from the instructor:

Notifications related to this class will be posted in Canvas under Announcements (set up your Canvas notifications properly).

Classroom Behavior:

The main rule is to not disrupt or distract other students during class! Do not hold conversations with neighbors during class. Additionally please treat other students and the instructor and TA with respect.

Accommodations for students with disability:

Texas State University is committed to providing equitable access to learning opportunities for all students. If you experience barriers to your education due to a disability or think you may have a disability, please contact Disability Services (DS) at (512) 245-3451, by email at ods@txstate.edu, or visit the [Disability Services web page](#).

Disabilities may include, but are not limited to: attentional, learning, mental health, sensory, physical, communication, or chronic medical conditions. All students are encouraged to discuss their disability related needs with the DS staff and their instructors as soon as possible.

Students who qualify for extra time for exams must take their exams with [ATSD](#) and must schedule their exam at the same time the exam is given in class. You must submit your request with ATSD at least 3 business days before the exam!

Expectations:

I expect you to do the following, in order to succeed in the class:

- Do the zybooks activities, preferably before I go over the corresponding material in class (see daily announcements for topics covered each day). Keep up!
- Play with the sample code presented in class (it will be in zybooks)
- Do the programming assignments, spread out over several days
- Study (early) for the quizzes, exams, and final exam.

Academic Honesty:

You are expected to adhere to the University's Academic Honor Code:

<https://www.txst.edu/honorcodecouncil/Academic-Integrity.html>

- **All assignments must be done individually**, with the following exception:

Quizzes (except for quiz 0):

- Students will work together in pairs on the coding quizzes.
- No more than 2 may collaborate together during the quiz.

- **Do not include code obtained from the internet or any other source** (another student or an AI tool, or a lab tutor) **in your programming assignment**, except what is provided by the instructor during the current semester.
- **Do not post your programming assignment solution anywhere** on the internet, or share any code you have written for an assignment with anyone else.
- **Examinations must be your own work.** Talking or discussion is not permitted. Do not compare notes, copy from others, or collaborate in any way while you are still in the classroom. Do not access mobile devices or smart watches or glasses during an exam.
- The penalty for violating academic honesty on an assignment, quiz, test or exam is a reduction of points up to a grade of 0 on the assignment, quiz, test or exam.
- If you receive a 0 for cheating on an assignment, quiz, test or exam, I will follow the process described [here](#), which may lead to a hearing with the Texas State University Honor Code Council.

Artificial Intelligence (AI) Use in This Course

What Counts as AI

AI refers to any program or tool that can generate or edit content, including ChatGPT, Copilot, and AI features in development environments.

How You Can Use AI

You may use AI tools to support your learning, such as:

- Asking for explanations of C++ concepts or syntax
- Getting help understanding error messages
- Clarifying assignment requirements
- Learning about general programming approaches

Think of AI as a study partner. It's helpful for practice and review, but not a replacement for your own work.

Note: You are encouraged to use the AI provided by zylabs: "Help me with this test"

What's Not Allowed

- No AI-generated code: All code you turn in must be written and tested by you. Copying, adapting, or "tweaking" AI-generated code counts as prohibited use.
- No AI-submitted work: You cannot submit AI-generated writing or explanations as your own.
- No auto-complete coding from AI: If your IDE/editor has code generation features (e.g., Copilot), you need to turn them off or actively ignore them. You're responsible for ensuring your work is your own.

Why These Restrictions Exist

This course is about developing your own programming and problem-solving skills. If you rely on AI to write your code, you skip the practice you need to become a strong programmer. By writing your own solutions, you'll build skills that AI tools can't give you.

Your Responsibility

If you use AI in permitted ways, you must:

- Make sure you fully understand the explanations it provides
- Write your own original code from scratch
- Be able to explain and reproduce your solutions if asked

Misusing AI not only undermines your learning, but may also violate academic integrity policies. The goal is for you to leave this class confident in your own ability to solve problems and write programs independently.